# Design Patterns

## trong Phát triển Phần mềm

Phân tích và vận dụng các
Behavioral, Structural và Creational Patterns

**Nhóm 10**

23127006 – Trần Nguyễn Khải Luân
23127179 – Nguyễn Bảo Duy
23127404 – Lê Tuấn Lộc

Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

*Học phần:* Thiết kế phần mềm

Ngày 28 tháng 2 năm 2026

# Nội dung trình bày

# Issue #2 — Vi phạm DRY: Duplicate `productStatus` Logic

**Vấn đề:** Khối logic xác định trạng thái auction được copy nguyên vẹn vào 2 route handler khác nhau.

**Nguyên tắc vi phạm:** DRY (Don't Repeat Yourself)

## Business rule bị encode 2 lần

- `GET /detail` → 5-nhánh if/else (dòng ∼162)
- `GET /complete-order` → 4-nhánh if/else (dòng ∼984)
- Hai bản đã **drift**: bản thứ hai thiếu nhánh `ACTIVE` cuối

**Rủi ro kỹ thuật:**

- Thêm state `DISPUTED` → phải sửa **cả hai chỗ**; nếu quên một, hai route trả kết quả mâu thuẫn nhau với cùng một product
- Không có unit nào để test — logic embedded sâu trong HTTP

# Issue #2 — Refactoring: Extract `resolveAuctionStatus()`

**Chiến lược:** Extract Function → đưa về `services/auction/auction-state.js`

**Trước** — inline, lặp lại:

```
// Cùng khối xuất hiện 2 lần
const now = new Date();
const end = new Date(product.end_at);
let status = 'ACTIVE';
if (product.is_sold === true)
  status = 'SOLD';
else if (...)
  status = 'CANCELLED';
// ... 3 nhánh nữa
```

**Sau** — một hàm duy nhất:

```
// auction-state.js
export function
resolveAuctionStatus(product) {
  if (product.is_sold === true)
    return 'SOLD';
  if (product.is_sold === false)
    return 'CANCELLED';
  // ...
  return 'ACTIVE';
}

// Trong route handler:
const status =
```

# Issue #6 — Vi phạm OCP: State Taxonomy Không Tập Trung

**Vấn đề:** Logic phân loại state auction được viết trực tiếp trong route handlers dưới dạng if/else inline — không có module "đóng"bảo vệ core logic.

**Nguyên tắc vi phạm:** OCP (Open/Closed Principle)

## Phân biệt quan trọng

Guard condition (`if (!product) return 404`) **không** phải OCP violation.
State classification (`if (...) status = 'PENDING'`) **là** OCP violation — đây là *trục mở rộng domain*.

**Hậu quả khi thêm state `PAUSED`:**

- Phải **mở (modify)** code đang chạy trong production ở nhiều file

# Issue #6 — Refactoring: Encapsulate Variation (State Module)

**Chiến lược:** Tập trung toàn bộ state taxonomy vào một module duy nhất.
Module này **đóng** với modification, **mở** cho extension.

## `auction-state.js` — Single Point of Extension

- `resolveAuctionStatus(product)` → trả về state string
- `canBid(status)` → guard function
- `canAccessOrder(status)` → guard function
- `requiresAccessCheck(status)` → guard function

**Thêm state `PAUSED` sau refactor:**

1. Thêm 1 điều kiện vào `resolveAuctionStatus()` trong `auction-state.js`

# Issue #14 — Vi phạm DRY: `auctionEndNotifier` Tự Tính State

**Vấn đề:** `auctionEndNotifier.js` — module chạy độc lập — tự encode business rule về auction state theo cách riêng, tạo ra **nguồn sự thật thứ ba**.

**Nguyên tắc vi phạm:** DRY ở cấp *cross-module knowledge duplication*

## Ba nguồn sự thật trong cùng một hệ thống

- `GET /detail`: 5-nhánh if/else (kiểm tra `is_sold` + `highest_bidder_id`)
- `GET /complete-order`: 4-nhánh if/else (đã drift)
- `auctionEndNotifier.js`: `if (auction.highest_bidder_id)` — 2-nhánh, **không kiểm tra `is_sold`**

# Issue #14 — Refactoring: Notifier Depend vào `auction-state.js`

**Chiến lược:** Notifier không tự tính state — import và dùng `resolveAuctionStatus()`.

**Trước** — implicit, thiếu check:

```
// implicit state check, 2 nhánh
// không check is_sold
if (auction.highest_bidder_id) {
  await sendMail(winner, ...);
  await sendMail(seller, ...);
} else {
  await sendMail(seller, ...);
}
```

**Sau** — explicit, đúng rule:

```
import { resolveAuctionStatus }
  from '../services/auction/
    auction-state.js';

const status =
  resolveAuctionStatus(auction);

if (status === 'PENDING') {
  await notifyWinner(auction);
  await notifySeller(auction);
} else if (status === 'EXPIRED') {
  await notifySellerNoBid(auction);
}
```

# Issue #11 — Vi phạm DRY + ISP: Dual INSERT API

**Vấn đề:** `review.model.js` export **hai hàm** thực hiện cùng một INSERT vào bảng `reviews`, với **field naming convention khác nhau**.

**Nguyên tắc vi phạm:** DRY + ISP

## Hai hàm, hai convention — cùng một DB operation

- `createReview(reviewData)` → caller dùng `reviewee_id` [*seller.route.js*]
- `create(data)` → caller dùng `reviewed_user_id` [*product/account.route.js*]
- Cả hai đều INSERT vào cùng column `reviewee_id` trong DB

**Rủi ro kỹ thuật:**

# Issue #11 — Refactoring: Một API duy nhất, camelCase nhất quán

**Chiến lược:** Merge thành một hàm canonical với destructuring params (camelCase).

**Trước** — 2 hàm, 2 convention:

```
// seller.route.js
await reviewModel.createReview({
  reviewer_id: sellerId,
  reviewee_id: bidder_id, // ← field A
  product_id: productId, ...
});

// product.route.js
await reviewModel.create({
  reviewer_id: reviewerId,
  reviewed_user_id: revieweeId, // ← field B
  product_id: ..., ...
});
```

**Sau** — 1 hàm, camelCase:

```
// review.model.js
export function createReview({
  reviewerId, revieweeId,
  productId, rating,
  comment = null }) {
  return db('reviews').insert({
    reviewer_id: reviewerId,
    reviewee_id: revieweeId, // ← 1
    product_id:  productId,
    ...
  });
}
// Tất cả caller dùng cùng interfac
```

# Issue #12 — Vi phạm DRY + YAGNI: Dual UPDATE API

**Vấn đề:** Hai hàm UPDATE khác nhau ở số lượng tham số WHERE — nhưng tham số thêm (`reviewee_id`) là **redundant predicate** về mặt domain.

**Nguyên tắc vi phạm:** DRY + YAGNI

## Domain invariant quan trọng

**(reviewer_id, product_id) đã đủ unique** — một người chỉ review một lần cho một sản phẩm.
Thêm `reviewee_id` vào WHERE không thu hẹp result set — nhưng nếu truyền sai, **UPDATE 0 rows silently**.

**Biểu hiện inconsistency trong `seller.route.js`:**

- `POST /products/:id/rate` → dùng

# Issue #12 — Refactoring: Encode Domain Invariant vào WHERE

**Chiến lược:** Giữ lại 2-param version, bỏ 4-param version, đổi sang camelCase.

**Trước** — 2 hàm, 2 WHERE:

```
// 4-param (seller.route PUT /rate)
await reviewModel.updateReview(
  sellerId,
  highest_bidder_id, // thừa
  productId, data
);

// 3-param (seller.route POST /rate)
await reviewModel
  .updateByReviewerAndProduct(
    sellerId, productId, data
);
// ← Cùng file, 2 hàm khác nhau
```

**Sau** — 1 hàm, WHERE đúng:

```
// review.model.js
export function updateReview(
  reviewerId, productId, updateData
) {
  return db('reviews')
    .where('reviewer_id', reviewerI
    .where('product_id',  productId
// (reviewer, product) = unique
    .update(updateData);
}

// Mọi caller đều dùng:
await reviewModel.updateReview(
```

# Tổng Kết Issues #11 & #12 — Model Layer Anti-Pattern

## Root Cause chung

**Incremental development** không có consolidation: mỗi lần thêm feature mới, một hàm mới được tạo thay vì mở rộng hàm hiện có → duplicate API tích lũy.

**Triệu chứng:**

- 2 INSERT functions: `create()` vs `createReview()`
- 2 UPDATE functions: 3-param vs 4-param
- Caller buộc phải biết *convention nội bộ* của model

**Kết quả sau refactor:**

- 1 INSERT: `createReview({...camelCas`
- 1 UPDATE: `updateReview(userId, productId, data)`
- Domain invariant encoded trong WHERE clause

# Issue #10 — YAGNI + KISS: Dead Code Ẩn Sau `try/catch`

**Bối cảnh:** `GET /bid-history/:productId` là một **JSON API route**. Sau khi `try/catch` gọi `res.json()`, handler còn chứa ≈**30 dòng code không bao giờ chạy**.

**Nguyên tắc vi phạm:** YAGNI + KISS

## Ba vấn đề cộng hưởng trong cùng dead code block

1. **Scope error:** `productId` được khai báo bằng `const` *bên trong* `try {}` — dead code bên ngoài reference nó → `ReferenceError` nếu được kích hoạt.
2. **Duplicate object key:** `seller_id` được khai báo **hai lần** trong object literal → JS silently discard key đầu tiên — latent data bug.
3. **Double response:** `res.render()` được gọi sau `res.json()`

# Issue #10 — Refactoring: Delete, Don't Patch

**Chiến lược: Pure deletion** — không thêm abstraction, không restructure, chỉ xóa dead code.

**Trước** — 42 dòng, 2 response path:

```
router.get('/bid-history/:id',
  async (req, res) => {
  try {
    const productId = parseInt(...);
    const history = await ...;
    res.json({ success: true,
               data: history }); } catch (err) {
  } catch (err) { ... }

  // ← DEAD: productId out of scope
  const result = await productModel
    .findByProductId(productId);
  const product = {
    seller_id: result[0].seller
```

**Sau** — 9 dòng, rõ ràng:

```
router.get('/bid-history/:id',
  async (req, res) => {
  try {
    const productId = parseInt(...);
    const history = await ...;
    res.json({ success: true,
               data: history });
  } catch (err) {
    res.status(500).json({ ... });
  }
});// ← Clean. Done.
```

**Kết quả đo lường:**

# Issue #4 — SRP + KISS: `POST /bid` là God Handler ( 430 dòng)

**Vấn đề:** Một handler thực hiện **15 bước liên tiếp**, không có bước nào được đặt tên hay tách hàm.

**Nguyên tắc vi phạm:** SRP + KISS

## Trong một function duy nhất, code thực hiện đồng thời

**Domain concerns:**

- 5 eligibility checks
- Bid amount validation
- 4-case proxy bidding algorithm
- Buy-now price enforcement

**Infrastructure concerns:**

- DB transaction + row lock
- 3 table writes
- 3 email HTML bodies
- Parallel email dispatch

# Issue #5 — OCP: Thuật Toán Proxy Bidding Bị Nhúng Vào Route

**Vấn đề:** 4-case `if/else` cho thuật toán đấu giá tự động nằm **trực tiếp trong HTTP handler**, không được đặt tên hay cô lập.

**Nguyên tắc vi phạm:** OCP

## Invariants của thuật toán — không có tên, không có documentation

1. **Self-bid:** Nếu user đã là highest bidder $\rightarrow$ chỉ update max price, không đổi current price
2. **First bid:** Chưa có ai đặt $\rightarrow$ giá hiện tại = starting price
3. **New bid** $<$ **existing max:** Người cũ thắng, giá = new bid amount
4. **New bid** $\geq$ **existing max:** Người mới thắng, giá = old max + step

# Issue #8 — DIP: Route Phụ Thuộc Trực Tiếp Vào Infrastructure

**Vấn đề:** HTTP handler (high-level) import và gọi trực tiếp **concrete implementations** (low-level).

**Nguyên tắc vi phạm:** DIP

**Route biết quá nhiều về infrastructure:**

```
// product.route.js
import db from '../utils/db.js';          // Knex
import { sendMail } from '../utils/mailer.js'; // Nodemailer
// ...
await db.transaction(async (trx) => {
  await trx('products').forUpdate().first();
  await trx('bidding_history').insert({...});
  await trx.raw('INSERT INTO auto_bidding...');
});
```

**Hậu quả:**

- Đổi Knex → Prisma: **phải sửa route**
- Đổi Nodemailer → SendGrid: **phải sửa route**
- Unit test POST /bid: cần **live PostgreSQL**
- HTTP layer bị **coupled**

# Issues #4, #5, #8 — Giải Pháp: 3-Layer Separation

**Chiến lược:** Tách theo **trách nhiệm**, không phải theo feature. Giữ behavior hoàn toàn nguyên vẹn.

**`product.route.js`**
*HTTP Layer*

- Parse request
- Call service
- Send response

**22 dòng** (từ 430)

**`auction.service.js`**
*Orchestration Layer*

- Fetch pre-req data
- Run bid-engine (pure)
- DB transaction + writes
- Fire-and-forget emails

Owns all I/O

**`bid-engine.js`**
*Domain Layer*

- Validate eligibility
- Validate amount
- Auto-extend logic
- **Proxy-bid algorithm**

# Tổng Kết Issues #4, #5, #8 — Testability là Kết Quả

## Nhận thức cốt lõi

Testability là *hệ quả tự nhiên* của đúng SRP. Khi mỗi function có một trách nhiệm duy nhất, nó tự nhiên trở nên unit-testable.

**Trước refactor:**

- 0 pure function trong bidding flow
- Unit test `POST /bid` cần live DB
- Proxy-bid invariants: ẩn trong inline code
- Thêm bid type: sửa 430-line handler

**Sau refactor:**

- **6 pure functions** trong `bid-engine.js`
- Unit test algorithm: mock object duy nhất
- Invariants: documented + named
- Thêm bid validation: sửa `bid-engine.js`

# Tổng Kết: Ba Issues, Một Giải Pháp

## Root Cause chung

Không có *single source of truth* cho auction state logic.
Issues #2, #6, #14 là cùng một vấn đề nhìn từ ba góc độ khác nhau.

**Trước refactor:**

- 3 nơi encode cùng 1 rule
- Drift đã xảy ra giữa các bản sao
- Không thể unit test
- Thêm state $\rightarrow$ sửa $\geq 3$ file

**Sau refactor:**

- 1 module: `auction-state.js`
- Pure function, zero infrastructure dependency
- Test: `resolveAuctionStatus(mock` trực tiếp
- Thêm state $\rightarrow$ sửa **1 file**

*"Don't repeat yourself" không chỉ về code — về **knowledge**: mỗi*

# Issue #9 — SRP + DIP: Order Được Tạo Sai Nơi & Sai Lúc

**Vấn đề:** Order entity chỉ được tạo **lazily** khi user navigate tới `GET /complete-order` — không phải tại thời điểm auction kết thúc.

**Nguyên tắc vi phạm:** SRP + DIP

## Ba code path kết thúc auction — không path nào tạo Order

- `executeBuyNow()` → đóng auction, **không tạo Order**
- `placeBid()` khi buy-now triggered → đóng auction, **không tạo Order**
- `auctionEndNotifier.js` khi hết hạn → gửi email, **không tạo Order**

## SRP Violation: GET handler có write side effect

# Issue #9 — Refactoring: order.service.js + Đúng Timing

**Chiến lược:** Tạo Order **tại thời điểm auction transition sang PENDING** — không phụ thuộc vào route nào user visit. Idempotent để tránh duplicate.

**Trước** — lazy, fragile:

```
// GET /complete-order (route handler)
let order =
  await orderModel.findByProductId(
if (!order) {
  // Business logic in HTTP layer!
  await orderModel.createOrder({
    product_id, buyer_id,
    seller_id, final_price
  });
  order =
    await orderModel.findByProductId(
```

**Sau** — domain-driven, idempotent:

```
// order.service.js (NEW)
Export) async function
  createOrderFromAuction(
    { productId, buyerId,
      sellerId, finalPrice }) {
  const existing = await
    orderModel.findByProductId(prod
  if (existing) return existing; //

  return orderModel.createOrder({ .
}
```